Express Car Complete Code Explanation

Viva / exam reference document based on the current Flutter project codebase. It explains what each major file, service, package, and flow is used for.

1. App Purpose

- Express Car is a car-rental application built with Flutter.
- It uses Firebase for authentication, data storage, and car/booking management.
- It also supports image upload, profile management, booking, and live location tracking.
- The app has different flows for normal users, admins, and car owners.

2. App Start Flow

- **main.dart** initializes Flutter bindings, Firebase, and app theme.
- After Firebase initialization, it calls **fetchCarsFromFirestore()** once to preload car data.
- The app starts with **SplashScreenWrapper**, waits briefly, and then navigates to **AuthWrapper**.
- **AuthWrapper** decides the next screen depending on authentication state, role, and profile completion.

3. Authentication Flow

- **signup_page.dart** creates a new account using email/password or Google Sign-In.
- After signup, it creates a Firestore document under the **users** collection with default role **user**.
- **signin_page_new.dart** signs the user in and sends them back to **AuthWrapper** so the app can decide which page to open.
- **CompleteProfile.dart** is used when required profile fields are missing.
- **admin_login_page.dart** is used for admin access to the admin panel.

4. Role-Based Routing

- If the logged-in user role is **admin**, the app opens **AdminPanelPage**.
- If the profile is incomplete, the app opens **CompleteProfilePage**.
- If everything is complete and role is normal user, the app opens **HomePage**.
- This logic is centralized in **auth_wrapper.dart** so navigation stays consistent everywhere.

5. Image Handling - Full Explanation

- **Add_Car.dart** allows the admin or owner to choose a car image from gallery or from local assets.
- **image_picker** is used to open the phone gallery and select the image.
- The image is uploaded through **imgbb_upload_service.dart** first, using the ImgBB REST API.
- If ImgBB fails, the app falls back to **Firebase Storage** so the car can still be saved.
- The final uploaded image URL is stored in Firestore in the **image_url** field.
- **car_image_widget.dart** is the main reusable image display widget.
- It normalizes unsafe or malformed image URLs using **normalizeApiImageUrl()**.
- It shows **Image.network** for valid URLs, a loading spinner while loading, and a local asset fallback when the network image fails.
- **car_model.dart** parses the image field from Firestore and also provides fallback asset images from **assets/images**.

6. Why Image Code Is Used

- To make car listings visible in the home page, booking pages, favorite pages, and admin pages.
- To avoid broken UI when remote image URLs are invalid.
- To ensure there is always a fallback car image from the assets folder.
- To let the app save a car even if ImgBB is not responding.

7. Location Tracking - Full Explanation

- **geolocator** is used to request permission and read the device GPS position.

- **car_location_tracking_service.dart** is the main singleton service for live tracking.
- It starts a position stream using high accuracy and a distance filter.
- For the logged-in car owner, it finds owned cars using **owner_email** or **owner_uid**.
- It then batch-updates Firestore documents in both **cars** and **fleet_items** collections.
- Each update stores **current_location** as a GeoPoint and also saves latitude, longitude, accuracy, speed, heading, and timestamp fields.
- **home_page.dart** starts this service so tracking remains active while the app is open.
- **live_booking_tracking_page.dart** reads those Firestore location fields and shows them on a Flutter Map.

8. Why Location Code Is Used

- To show the live position of a car on the map.
- To let users see their own position while tracking a booking.
- To store live tracking data in Firestore so it can be reused across pages.
- To make driver/owner tracking visible in real time.

9. Booking Flow

- **Book_car.dart** handles the booking form and booking submission.
- It stores booking data in Firestore under the **bookings** collection.
- **booking_details_page.dart** shows booking summary, car image, rental type, price details, and driver document info.
- **live_booking_tracking_page.dart** is opened from booking details for live map tracking.
- Driverless booking uses extra documents such as Aadhar, license, and PAN details.

10. Admin and Owner Pages

- **AdminPanelPage** is the admin dashboard.
- **HandleBussiness.dart** is the main owner/admin action hub for Add Car, Manage Cars, and Manage Bookings.
- **Manage_Cars.dart** shows cars owned by the current user and allows delete/edit actions.
- **Manage_Booking.dart** shows bookings for the owner?s cars.
- **fleet_driver_manager.dart** is used for fleet/driver document and image handling, including Cloudinary uploads.

11. Important Firestore Collections

Collection	Use
users	Stores user profile, role, favorites, presence, and profile data
cars	Stores car listings, images, owner info, and live location
fleet_items	Alternate live-vehicle collection updated by tracking service
bookings	Stores booking records, payment details, and documents
counters	Used for unique car ID generation

12. Packages Used and Their Purpose

Package	Purpose
firebase_core	Initialize Firebase
firebase_auth	Sign in and sign up users
cloud_firestore	Store and read app data
firebase_storage	Fallback image storage
google_sign_in	Google login support
image_picker	Pick images from the phone gallery
http	Send ImgBB upload requests
geolocator	Fetch GPS location and permissions

flutter_map	Show tracking map
latlong2	Create latitude/longitude points
cloudinary_public	Upload documents/images for admin flow
pinput	OTP input UI
multi_select_flutter	Select multiple car features

13. File-by-File Viva Explanation

- **main.dart** - App entry point, Firebase initialization, splash routing, and theme setup.
- **auth_wrapper.dart** - Checks auth state, role, and profile completeness; routes to the correct screen.
- **signup_page.dart** - User registration, Google signup, and Firestore user document creation.
- **signin_page_new.dart** - User login and redirection to AuthWrapper.
- **CompleteProfile.dart** - Collects missing profile details such as phone, address, DOB, and gender.
- **Add_Car.dart** - Add car form, image picking, image upload, GPS capture, and Firestore save.
- **imgbb_upload_service.dart** - Reusable ImgBB upload helper with error handling and API key support.
- **car_image_widget.dart** - Network image normalization and safe image display with fallback asset.
- **car_model.dart** - Car model parser for Firestore data and fallback assets.
- **car_location_tracking_service.dart** - Live GPS tracking service that updates car documents in Firestore.
- **home_page.dart** - User car listing page and tracking start point.
- **Book_car.dart** - Booking form and booking save flow.
- **booking_details_page.dart** - Shows booking details, image, pricing, and live tracking button.
- **live_booking_tracking_page.dart** - Map page that shows car and user position live.
- **Manage_Cars.dart** - Owner car list management, delete, and edit actions.
- **Manage_Booking.dart** - Owner booking overview for owned cars.
- **HandleBussiness.dart** - Owner/admin action dashboard with shortcuts to car management pages.
- **AdminPanel.dart** - Admin dashboard for managing the whole system.
- **fleet_driver_manager.dart** - Fleet/driver uploads and Cloudinary-based document handling.
- **EditProfile.dart** - Edit user profile information and photo upload.
- **Favorite.dart** - Displays favorite cars and image cards.

14. Important Fields Used in Firestore

- Car fields: **car_id**, **owner_email**, **owner_uid**, **name**, **model**, **type**, **number_plate**, **features**, **price_per_day**, **price_per_week**, **price_per_month**, **image_url**, **current_location**, **location_lat**, **location_lng**, **location_updated_at**.
- User fields: **displayName**, **email**, **photoURL**, **role**, **favorites**, **phone**, **address**, **dob**, **gender**.
- Booking fields: **booking_id**, **car_id**, **car_name**, **image_url**, **start_date**, **end_date**, **pickup_location**, **total_amount**, **fleet_type**.

15. Short Viva Answer

This Flutter app is a Firebase-based car rental system. Firebase Auth handles login, Firestore stores users, cars, and bookings, ImgBB and Firebase Storage handle image uploads, and Geolocator with Flutter Map handles live location tracking. AuthWrapper controls the role-based routing, Add_Car.dart manages image and location saving, and live_booking_tracking_page.dart shows live car location on the map.

Prepared from the current workspace codebase on 21 April 2026.